eMudhra CA key compromise: enterprise response

What we need from service owners, and how we close the long tail

Senior IT management and service owners · 26 October 2026

Draft for discussion: no decisions taken, no enterprise assessment completed

EXECUTIVE SUMMARY

Browsers are protected; the remaining risk sits in software that keeps its own trust list

Three things to know, and five decisions requested this week

WHAT HAPPENED

- Eight eMudhra intermediate CA keys were weak from generation in April 2025 and can be recovered by anyone.
- Six sign TLS certificates and two sign code. Anyone with a key can forge certificates that unprotected clients accept.

WHERE WE STAND

- Vendors blocked the keys in browsers and OS trust stores on 17–20 Oct, with no user action. That holds only under four enterprise conditions.
- Java, Python, Node, Go, containers, appliances and agents stay exposed until updated.

WHAT WE ASK

- Every service returns one six-field record, worked in priority order.
- Fix the inspection proxy, identity paths and build pipelines first.
- Replace our certificates under emSign ECC Root CA - G3: Chrome drops it on 27 Oct.

DECISIONS THIS WEEK D-01 prioritisation model · D-02 browser proviso · D-03 target windows · D-04 replace our emSign certificates · D-07 inspection-proxy policy

Sources: Google Security Blog; Mozilla; BleepingComputer; CSO Online; Technobezz (20–23 Oct 2026). Package facts checked directly on 26 Oct (PyPI, crates.io, Go proxy, Ubuntu changelog).
Detail: README.md, Situation as at 26 October.

WHAT HAPPENED

The keys were weak for 18 months, and vendors closed the browser gap in four days

Exposure ran from key generation until each client was protected, and it is still open for unpatched software

Exposure window | **Vendor response**

535 days: keys recoverable by anyone



30 Apr 2025
Weak keys generated at a key ceremony

17 Oct
Reported. Chrome CRLSet and Firefox OneCRL block the same day

19 Oct
Apple; Android via Play services; Sectigo revokes AAA cross-certificates; Go fix

20 Oct
Microsoft; Ubuntu ca-certificates. Node, Debian still pending

27 Oct
Chrome removes emSign ECC Root CA - G3: legitimate sites under it break

8
intermediate keys: 6 TLS, 2 code-signing

0
confirmed misuse reported by 21 Oct. Clients that don't check CT can't rule it out

~30,000
legitimate certificates to migrate (eMudhra estimate)

Sources: Google Security Blog; Mozilla; BleepingComputer; CSO Online; Technobezz (20–23 Oct 2026). Package facts checked directly on 26 Oct (PyPI, crates.io, Go proxy, Ubuntu changelog).
Detail: README.md, Situation as at 26 October.

Exposure depends on what a system trusts, not on whose certificates we buy

The verifier decides. Four kinds, from protected to exposed

A Browsers and OS trust stores

Chrome, Edge, Firefox, Safari; Windows and Apple stores; Android

Protected by vendor push. Confirm coverage

B OS-store consumers

.NET on Windows, curl, PHP, Ruby, Go on Linux using system roots

Protected once the OS update is confirmed

C Own or compiled trust list

Java cacerts, Python certifi, Node built-in roots, Go/Rust binaries, container images

Exposed until updated or rebuilt

D Embedded or vendor-controlled

Appliances, agents, packaged products, OT/IoT, SaaS

Exposed until the vendor fixes it



TLS-inspection proxy: it is the verifier for every client behind it.
Browsers only see the proxy's re-signed certificate, so their blocks never see the real chain. Fix the proxy once and every inspected client is protected on that path, including non-browser software.

An attacker needs to intercept or redirect traffic, except for inbound mTLS, where a forged client certificate connects directly.

Browsers count as protected only when four enterprise conditions hold

We record them as "Validated" from a coverage report, not as "fixed" by assumption

Platform	Mechanism	Date
Chrome	CRLSet block	17 Oct
Firefox	OneCRL block	17 Oct
Apple	Trust block	19 Oct
Android	Google Play services	c. 19 Oct
Windows / Edge	Microsoft block	20 Oct
Chrome	Root removal (availability)	27 Oct

1 Reachability

Devices online since 20 Oct; nothing blocks browser component updates, Remote Settings or Windows root auto-update.

2 Currency

Supported browser and OS versions; Android Play services current.

3 No interception

Traffic not decrypted by a TLS-inspection proxy, or the proxy has passed PB-09.

4 Image hygiene

VDI golden images, kiosks and long-offline devices refreshed or checked.

Not covered by the proviso: other software on the same desktop. That includes CLI tools (macOS's built-in curl still accepted the chain after Apple's block), Java, Python, Node and Go apps, Electron apps, VPN/ZTNA and management agents, and developer toolchains. These belong to the long tail.

Sources: Google Security Blog; Mozilla; BleepingComputer; CSO Online; Technobezz (20–23 Oct 2026). Package facts checked directly on 26 Oct (PyPI, crates.io, Go proxy, Ubuntu changelog).
Detail: README.md, Situation as at 26 October. Not covered: macOS curl result from lilting.ch test, 21 Oct.

The long tail is where the remaining work and risk sit

Phase 1 confirms vendor coverage. Phase 2 finds, fixes or contains everything that keeps its own trust list

Phase 1 · Vendor-protected: confirm coverage	Phase 2 · Long tail: find, fix or contain, prove
Server-side API clients	Runtimes with own bundles
Containers and serverless	CI/CD and artifact proxies
Desktop non-browser software	Network and security appliances
VPN, ZTNA and agents	OT, IoT and facilities
In-house mobile apps	Suppliers, SaaS and inbound mTLS

Find it without a CBOM

1. **Egress telemetry**
   30 days of outbound TLS from proxy, firewall, NAT and cloud egress logs, by source.
2. **Map to owners**
   Each source becomes a long-tail record (workbook sheet).
3. **Direct-to-internet = finding**
   TLS that bypasses the proxy has no shared control.
4. **Add inventories**
   Software inventory, container registry, supplier register.

Discovery uses egress telemetry we already hold. No CBOM, new scanner or full certificate inventory is required (pack principle retained).

Every system gets two ratings: how bad a forged certificate would be, and whether it would be accepted

Priority combines the two (next slide). Full definitions: RISK_TIERS.md sections 2 and 3

① Impact

If this system accepted a forged certificate, what could an attacker do?

H — **Credentials, sensitive data, privileged actions, code execution or many systems**
e.g. payment and identity integrations, build pipelines, inspection proxies, admin tooling

M — **Business data or service integrity, with a limited blast radius**
e.g. general API clients, internal service-to-service TLS, desktops

L — **Little consequence, or a demonstrably limited role**
e.g. serve-only websites, private-CA-only services

② Verifier susceptibility

Would the software that checks the certificate accept the forged chain today?

	Type	Typical examples	Status today
A	**Vendor already blocks it**	Chrome, Edge, Firefox, Safari; Windows/Apple stores; Android	**Protected if the proviso holds**
B	**OS store; patch not yet confirmed**	.NET on Windows, curl, PHP, Ruby, Go on Linux	**Safe once the OS update is confirmed**
C	**Carries its own trust list**	Java cacerts, certifi, Node, Go/Rust binaries, containers	**Exposed until updated or rebuilt**
D	**Vendor-controlled; no fix yet**	Appliances, agents, packaged products, OT/IoT, SaaS	**Exposed: needs a compensating control**
?	**Unknown: not yet assessed**	Runtime or trust store not yet identified	**Assess first; unknown is not low**

Behind a TLS-inspection proxy, rate the path by the proxy's susceptibility: the proxy makes the trust decision. Version 1 tiers map to impact: Tier 1 → H, Tier 2/3 → M, Tier 4 → L.

Priority = impact × susceptibility: where common systems land

Each cell shows the priority and a typical example. Work P1 first; the vendor-blocked column only needs coverage confirmed

	A · Vendor already blocks it	B · OS store, patch unconfirmed	C · Own trust list	D · Vendor-controlled, no fix	Unknown · Not yet assessed
H Credentials, data, code execution	**P3** .NET payment client; Microsoft block confirmed	**P1** API validating JWTs; OS update unconfirmed	**P1** Batch job sending customer data to SaaS (Java)	**P1** Inspection proxy awaiting vendor fix: add exception	**P1 assess** Payment API; runtime not yet identified
M Business data, limited reach	**P4** Managed browser, not TLS-inspected	**P2** Internal service using the Linux OS store	**P2** Internal service using certifi or Node roots	**P2** Appliance cloud connector; track vendor	**P2 assess** Internal app not yet assessed
L Low consequence	**P4** Kiosk browser	**P3** Low-impact script using system curl	**P3** Low-impact tool with its own bundle	**P3** Printer firmware update check; track vendor	**P3 assess** Low-impact system not yet assessed

What the priority means

- **P1** — Contain in 72 h; close or except in 7 days
- **P2** — Close or except in 14 days
- **P3** — Close in 30 days
- **P4** — Confirm coverage in the normal cycle

"assess" = not yet rated: assess first, at that priority.

Route C (not applicable) sits outside the grid: serve-only sites, private-CA-only services.

Suspected exploitation escalates immediately.

Worked cases: RISK_TIERS.md sections 4 and 6. Windows are indicative pending decision D-03 (approval of the model: D-01).

FIX FIRST

Seven paths turn a forged certificate into stolen credentials or code execution

Treat these as P1 wherever the verifier is not yet confirmed



PB-09

Inspection and egress proxies

Decide trust for every client behind them. Block untrusted upstream certificates (D-07).



PB-16

Identity: JWKS, SAML metadata, tokens

A forged JWKS endpoint lets an attacker mint tokens your API accepts.



PB-08

Build pipelines and artifact proxies

Artifactory/Nexus pull for every build. Usually Java: fix the JVM store.



PB-04–07

Payment and customer-data APIs

Outbound calls from Java, Python, Node and Go services carrying sensitive data.



PB-16

Inbound mTLS trusting public roots

No interception needed: the attacker connects directly with a forged client certificate.



PB-12

Code signing and application control

Publisher-name rules could accept forged signatures while Windows trusted the chain.



PB-01

Admin and developer workstations

CLI tools and SDKs holding cloud credentials or signing keys sit outside the browser proviso.

AVAILABILITY · TUESDAY 27 OCTOBER

Chrome stops trusting emSign ECC Root CA - G3 tomorrow, and legitimate sites will break

A separate track from the security tiers, with its own owner and date

27 Oct

Chrome Root Store drops emSign ECC Root CA - G3

~30,000

legitimate certificates to migrate worldwide (eMudhra estimate)

Our certificates

- Find: lifecycle tool, CT search of our domains, endpoint scan
- Replace now with another public CA (D-04)
- Watch for appliances and pinned clients

PB-13

Partners we call

- Root removal in certifi ≥ 2026.10.18, webpki-roots ≥ 1.0.10 and Ubuntu breaks their emSign chains
- certifi 2026.10.23 also drops emSign C1 and ECC C3
- Time-boxed exceptions only (D-05)

PB-13

Service desk

- Knowledge article live before 27 Oct
- Issuer says eMudhra/emSign: route by owner
- Never advise bypassing warnings

PB-15

Sources: Google Security Blog (root removal, ~30,000 estimate). certifi and webpki-roots contents checked 26 Oct on PyPI and crates.io.

RETROSPECTIVE HUNT

Any certificate chaining to these keys is presumptively forged, which makes hunting cheap

eMudhra says the affected intermediates issued no end-entity certificates, so a single hit is an incident

535 days

from key generation (30 Apr 2025) to the first block (17 Oct 2026), and longer for unpatched clients

Code signing is the highest-value hunt: endpoint signer telemetry often has the longest look-back.

Hunt	Data source	Look for
H1	EDR file-signature telemetry	Binaries signed under the two code-signing intermediates
H2	TLS-inspection / SWG logs	Upstream certificates issued by the six TLS intermediates
H3	NDR / Zeek x509	Same issuers. TLS 1.3 hides certificates from passive sensors
H4	Application TLS logs (P1)	Peer certificate details on critical integrations
H5	Certificate Transparency	Our domains. Google found none before the block
H6	IdP and API logs	Unexpected JWKS key IDs or SAML signing changes

*Exclude *.we-have-a-problem.com: staff testing generates legitimate hits.*

Advanced Hunting retention is about 30 days; SIEM and archive retention varies. Record the gap to 30 Apr 2025 as a documented limitation (D-08).

Every service returns one six-field record and closes by one of three routes

Shared platform evidence is reused: applications reference it rather than repeat it

Six fields

1. Scope and owner
2. Priority and rationale
3. Trust configuration
4. Status and action
5. Evidence reference
6. Outstanding work and date

A — Standard platform assurance
Uses a centrally remediated platform with no custom trust. Evidence: assurance record + deployment coverage + “no overrides”.

B — Application-specific validation
Custom bundles, embedded runtimes, critical paths. Evidence: fix, activation and a test through the real client.

C — Not applicable to the assessed role
Serve-only or private-CA-only, with evidence. “We don't buy eMudhra certificates” is not a reason.

Unknown ≠ low · Clean scan ≠ protected · Deleting a cached intermediate ≠ distrust · Adding CAs never removes trust

From ACTION_PLAN.md section 4 and COMPLETION_TRACKER.md. Tracker columns are in the workbook's Service tracker sheet.

Each team owns its shared components, and applications point at that evidence

First actions for the next 72 hours (to 29 October)

Team	First action (72 h)	Playbook
Network / security engineering	Validate inspection proxy; set block on untrusted upstream; share issuer-log query	PB-09
PKI / certificate management	Replace our emSign ECC G3 certificates; publish test matrix and certificate reference	PB-13, PB-00
IAM / API platform	Assess JWKS, SAML metadata and inbound mTLS trust lists	PB-16
Container platform / DevSecOps	Rebuild base images; fix artifact-proxy JVM; list workloads by digest	PB-08
Linux and Windows platform	Publish Route A records per build; distribute disallowed certs where auto-update is off	PB-02, PB-03
Desktop / EUC and mobile	Coverage report against the four proviso conditions; inventory non-browser apps	PB-01, PB-10
Application owners	Identify runtime and trust store; P1 paths first; return the six-field record	PB-04 to PB-07
Endpoint security	Check application-control publisher rules; Windows distrust coverage	PB-12
SOC	Start hunts H1 and H2; record retention per source	PB-14
Supplier management / service desk	Requests to critical suppliers; knowledge article live	PB-11, PB-15

Map each functional team to a named owner and queue. No messages have been sent.

Seventeen playbooks, one format: applies if, fix, verify, record, escalate

Send delivery teams one document: playbooks/README.md, the team guide



Depth behind the playbooks: REMEDIATION_OPTIONS.md and APPLICATION_VALIDATION.md. Commands are illustrative and not yet run against our estate.

PLAN

Over the next 30 days, protect the chokepoints first, then work down the tail

Dates are indicative and depend on decision D-03

By 27 Oct
Today and tomorrow

- D-04 and D-16 decided
- Our emSign ECC G3 certificates replaced
- Service desk article live

By 29 Oct
72 hours

- D-01–D-03, D-05–D-07, D-12 decided
- Inspection proxy validated
- Identity paths assessed
- P1 contained
- Proviso coverage report
- Hunts H1–H2 run

By 2 Nov
7 days

- P1 closed or excepted
- Egress-led long-tail register
- Supplier requests sent
- Hunts H3–H6 done

By 9 Nov
14 days

- P2 closed or excepted
- Base images rebuilt and redeployed
- Vendor ETAs tracked

By 25 Nov
30 days

- P3 closed
- Long-tail cut-off (D-09)
- Closure review (D-14); P4 to business as usual

Throughout: daily updates only for open P1 items, blockers and outages; executive update twice weekly until P1 is closed (D-12). Facts rechecked daily.

Decisions requested

Eight of the sixteen in the workbook's Decisions sheet. The others are needed by 2 November

ID	Decision	Recommendation	Owner	By
D-01	Prioritisation model	Impact × verifier susceptibility; keep tier labels	Incident lead	28 Oct
D-02	Browser and desktop proviso	Accept vendor protection only with the coverage report	Head of EUC	28 Oct
D-03	Target windows	Indicative P1 72 h / 7 d · P2 14 d · P3 30 d	CISO	28 Oct
D-04	Our emSign ECC G3 certificates	Replace now; no extension for external endpoints	Head of PKI	27 Oct
D-05	Root removal in our stores	Accept partner impact; time-boxed exceptions only	Incident lead	29 Oct
D-06	AAA route on Windows and Java	Distrust specific intermediates; remove AAA only per vendor	Head of PKI	29 Oct
D-07	Inspection-proxy upstream policy	Block untrusted or revoked upstream certificates	Network security	28 Oct
D-08	Retrospective hunt scope	H1–H6 over available retention; record the gap	SOC lead	30 Oct

Package facts were checked directly; vendor actions come from press reports of vendor statements

Recheck daily while the incident is open (workbook Facts sheet)

Date	Fact	Confidence
30 Apr 2025	Eight keys generated with P-384 private values below 256	Secondary (citing Mozilla)
17 Oct	Chrome CRLSet and Firefox OneCRL block	Primary (Google)
19 Oct	Apple block; Android via Play services; Sectigo revokes AAA cross-certificates	Secondary
19 Oct	Go 1.27.3 and 1.26.10 published (rebuild required)	Verified release; secondary content
19–23 Oct	webpki-roots ≥ 1.0.10 and certifi ≥ 2026.10.18 drop emSign ECC Root CA - G3	Verified (package contents)
20 Oct	Microsoft block; Ubuntu 24.04 ca-certificates blocklists the root	Secondary; Ubuntu verified
26 Oct	Node.js 22.22.0 built-in roots still include the root	Verified
21 Oct	No confirmed misuse; one key public on GitHub; macOS curl still accepted chain	Secondary
27 Oct	Chrome Root Store removes emSign ECC Root CA - G3 (~30,000 certificates)	Primary (Google)

Primary vendor pages could not be fetched when version 2 was prepared. Confirm “Secondary” items against the vendor before relying on a detail. Sources: the References section of each document.